

Some CNN-based (non-transformer) architectures in classification for vision

Prof. Alexander Binder

August 18, 2025

Takeaway points

good practices in state of the art models (not only for vision!!!):

- residual connections/skip connections
- Batchnormalization or Layernorm
- use an ensemble of models rather than a single model
- stack smaller kernels rather than use a big kernel – partially weakened in ConvNext (kernels up to size 7).
- dropout regularization

The explanations for why these work well are often rather conceptual (except for ensembles).

Takeaway points

at the end of this lecture you should be able to summarize:

- VGG – as baseline
- resnet – specialties
 - residual connections
 - batchnorm
- densenet – specialties

Networks used for segmentation and GANs, image captioning and other tasks use similar building blocks

http://d2l.ai/chapter_convolutional-modern/index.html

- 1 Ye olde VGG
- 2 Dropout
- 3 ResNets and residual connections
- 4 Batch normalization
- 5 DenseNets
- 6 Ensembles
- 7 Closer to the state of the art



Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as “conv(receptive field size)-(number of channels)”. The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64	conv3-64	conv3-64	conv3-64
maxpool					
conv3-128	conv3-128	conv3-128	conv3-128	conv3-128	conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

Simonyan & Zisserman, ICLR 2015

<https://arxiv.org/abs/1409.1556>

- ◉ contribution: old-style network: repeated blocks of: (convolution-relu)* n -pooling
- ◉ only 3x3-convolutions to achieve larger fields of view by stacking
- ◉ 2014 ILSVRC competition second place.
- ◉ very large number of parameters: 130 millions!

Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as “conv(receptive field size)-(number of channels)”. The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64	conv3-64	conv3-64	conv3-64
maxpool					
conv3-128	conv3-128	conv3-128	conv3-128	conv3-128	conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

Simonyan & Zisserman, ICLR 2015

<https://arxiv.org/abs/1409.1556>

- very large number of parameters: 130 millions!
- 3 fully connected layers contain most of the parameters
- dropout layer for less overfitting between fc layers

- ① Ye olde VGG
- ② Dropout
- ③ ResNets and residual connections
- ④ Batch normalization
- ⑤ DenseNets
- ⑥ Ensembles
- ⑦ Closer to the state of the art

<http://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>

- Has one parameter: keep probability $p \in (0, 1]$.
- At test time, rescaled identity.

$$\text{eval mode: } \text{dropout}(\mathbf{x}) = p\mathbf{x}$$

- At training time: set randomly $1 - p$ of all neurons to zero. A way of adding noise to the learning problem.

$$\text{train mode: } \text{dropout}(\mathbf{x})[d] = \begin{cases} x[d] & \text{with probability } p \\ 0 & \text{with probability } 1 - p \end{cases}$$

why does it work?

Why does this work?

- two dimensions of the feature map $\phi_{d_1}(x)$ and $\phi_{d_2}(x)$ may have a correlation which helps to classify sample x on training data

example: 95% of all the time we have on training data $((x, y):$ input sample and its label) for a pair of two dimensions (d_1, d_2) :

$2\phi_{d_1}(x) - \phi_{d_2}(x) > 0$ whenever the label $y > 0$,

... **if** this correlation would not be not present in test data – then picking up such a correlation results in overfitting.

- setting $\phi_{d_1}(x)$ or $\phi_{d_2}(x)$ randomly to zero, prevents the algorithm from setting weights to use this non-generalizing correlation in a too strong way. it has to rely also on other correlations in the data.

One way to explain is:

- noise via setting features to zero reduces statistical correlation between features.
- Then the model cannot overfocus on a few correlations which are supported well by the training data
- instead it has to rely on a mixture of several different correlations between features. Some of them may not generalize / hold true for the source of your data P_{test} .

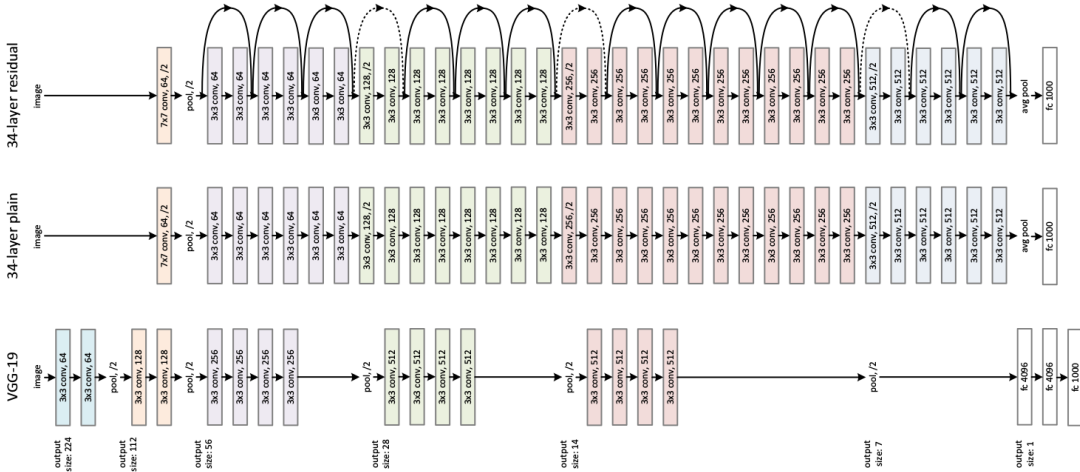
Inference Time Mistake !

- a model with dropout layers will behave in a randomized way, if the eval mode is not set!

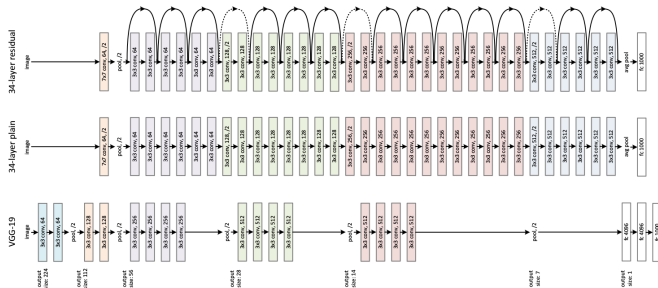
- Modern applications of Dropout: model-based uncertainty estimates,
example: Y Gal, Z Ghahramani <https://arxiv.org/abs/1506.02142>

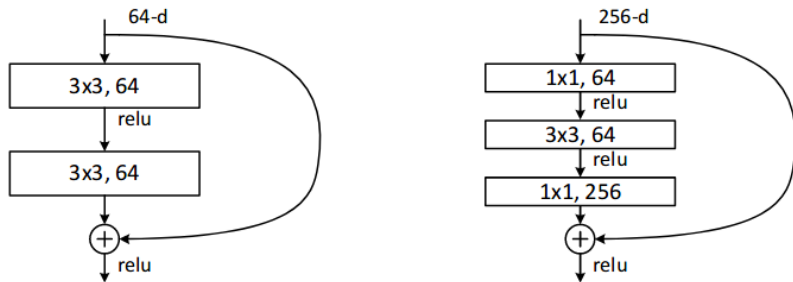
- ① Ye olde VGG
- ② Dropout
- ③ ResNets and residual connections
- ④ Batch normalization
- ⑤ DenseNets
- ⑥ Ensembles
- ⑦ Closer to the state of the art

<https://arxiv.org/abs/1512.03385>



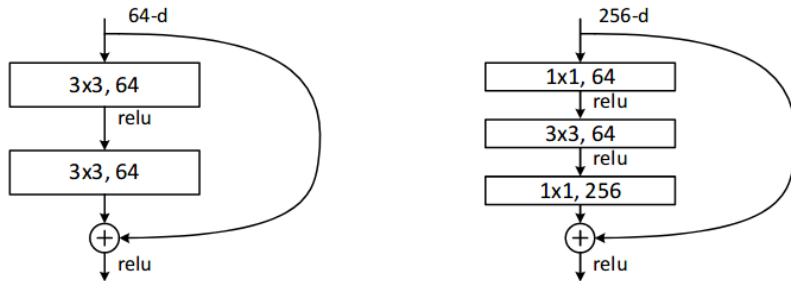
- ◉ <https://arxiv.org/abs/1512.03385>
- ◉ only 3×3 and 1×1 filters
- ◉ **main contribution 1: residual connections** – shortcuts across layers (usually as a linear mapping whenever number of filters changes, not as identity – see option B in Table 4)
- ◉ **main contribution 2: uses batchnormalization** after every convolution
- ◉ both contributions are about gradient flow
- ◉ in higher layers: once in a while half spatial size of feature maps, double number of filter channels





residual connection (2 conv blocks): $f(x) = x + C_1(C_2(x))$

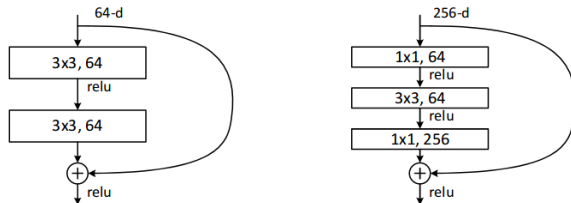
- add convolution-relu-convolution outputs on top of input feature map x



residual connection (2 conv blocks): $f(x) = x + C_1(C_2(x))$

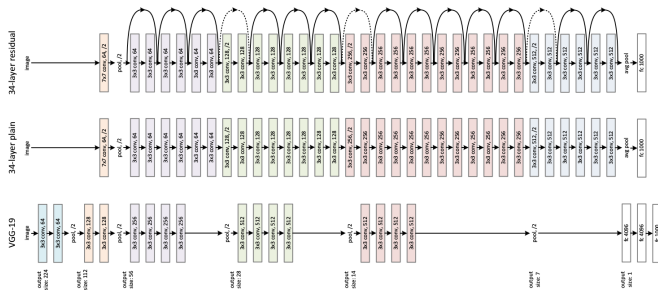
Why do residual connections work ?

- Reason 1 – gradient perspective: gradient flows as the identity through the shortcut, no vanishing gradient problem



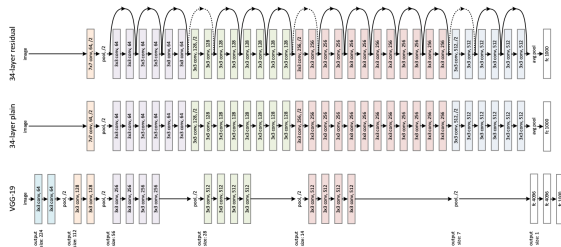
- Reason 2 – forward flow perspective: $f(x) = x + C_1(C_2(x))$
 - shortcut in forward pass inputs feature x from previous block
 - convolutions $C_2(C_1(x))$ across the residual path can learn additionally non-linear function on top
- option for quick unlearning to identity: if poor fit was learned during early phases of training in $C_2(C_1(x))$, it can be undone: update weights of convolution layers to zero, then get back the identity again.
(unhindered information flow fwd & gradient bwd)

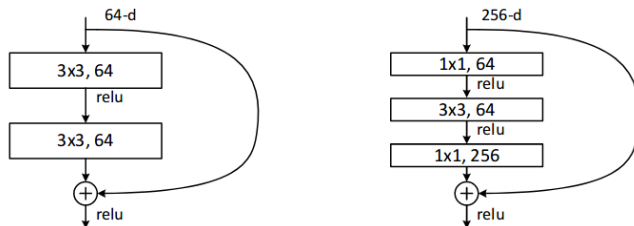
- identity + optional nonlinearity: $f(x) = x + C_1(C_2(x))$
 - allows to learn a more complex representation layer by layer
 - Network can start as: 1 conv layer and one fully connected layer, and (almost) only shortcuts in between.
 - Convolution kernels can add layer by layer some nonlinearities.



Why do residual connections work ?

- gradient flows as the identity through the shortcut, no vanishing gradient problem
- identity+ optional nonlinearity (the residual branch $C_1(C_2(x))$) $\Rightarrow$ convolutions across the parallel path can learn additionally non-linear function on top.
- option to unlearn non-linearities fast
- later on: compare to the memory cell in LSTM (1998) recurrent neural nets (RNNs)
<https://www.bioinf.jku.at/publications/older/2604.pdf> <https://arxiv.org/abs/1909.09586>. Earlier idea for gradient flow through time steps.





- Resnets use as second big change **batchnormalization** after every convolution before the ReLU

- ① Ye olde VGG
- ② Dropout
- ③ ResNets and residual connections
- ④ Batch normalization**
- ⑤ DenseNets
- ⑥ Ensembles
- ⑦ Closer to the state of the art

Ioffe and Szegedy, 2015 <https://arxiv.org/pdf/1502.03167.pdf>

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Batchnorm at training time performs 2 steps

- step 1: take one neuron , normalize its outputs so that – over the all elements in your minibatch have zero mean and standard deviation 1
- step 2: apply a simple affine transformation on the normalized output $y = \gamma \hat{x} + \beta$
- after this output has standard deviation γ and mean β , γ and β trainable

Ioffe and Szegedy, 2015 <https://arxiv.org/pdf/1502.03167.pdf>

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Batchnorm at training time performs 2 steps

- batchnorm at training time learns to output values which have constant mean and constant standard deviation (computed over the elements in a minibatch)
- convolution layers: compute mean, standard deviation, a, b not for one neuron and all samples in the minibatch but for all elements in the feature map of one channel in a conv layer and all samples in the minibatch – reduces parameters, treats each neuron in the same channel in the same way

Ioffe and Szegedy, 2015 <https://arxiv.org/pdf/1502.03167.pdf>

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1..m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Batchnorm at training time performs 2 steps

- batchnorm at training time learns to output values which have constant mean and constant standard deviation (computed over the elements in a minibatch)
- training time: update running mean and running variance for your training dataset (will be needed for test time)
- in order to work well requires usually a batchsize of 8 at least, better 16 or 32 or more.

Batchnorm at val/test time performs 2 steps:

- step 1: take one neuron, normalize its outputs by the running mean μ_{run} and running variance σ_{run}^2 learnt at training time (gets saved!).

$$\hat{x} = \frac{x - \mu_{run}}{\sqrt{\sigma_{run}^2 + \epsilon}}$$

- step 2: apply $y = a\hat{x} + b$. a, b the **trainable (!!)** rescaling parameters
- meaning: this simulates that the test sample would come from a very large batch with mean and variance statistics equal to the training data. under this assumption, any synthetic minibatch composed of many test samples would have mean b and std deviation a .
- **important/error source in coding:** use `model.eval()` or `model.train(False)` at testing time for your neural network, otherwise non-sense!

for convolution layers:

- one set of running mean, running variance, a , b scaling parameters are learnt for each channel.
- that is they are the same for all inputs/outputs of one channel

Why does batchnorm improve performance?

Equation on page 5 in the paper – Gradient with respect to inputs does not depend on scale of weight parameter anymore. Makes gradient flow more uniform across the channels of a layer.

See evidence for it in: Bjorck et al <https://arxiv.org/abs/1806.02375>.

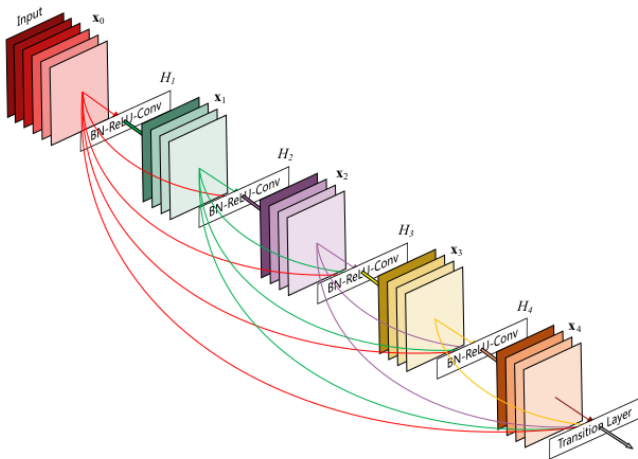
Layer normalization:

- Normalize for each sample separately.
 - statistics for normalization are computed over the spatial dimensions of a feature map $f[b, c, h, w]$ indexed by height h and width w
 - As if one would use batch normalization with a batch size of 1
- used in many Transformer architectures, for example in Swin Transformers, also in ConvNext
- the original Vision transformer uses group normalization

- ① Ye olde VGG
- ② Dropout
- ③ ResNets and residual connections
- ④ Batch normalization
- ⑤ DenseNets
- ⑥ Ensembles
- ⑦ Closer to the state of the art

Huang, Liu, van der Maaten, Weinberger, <https://arxiv.org/abs/1608.06993>

- resnets to the extreme: within a block of same feature map size (“dense block”), each layer contains the feature maps of each previous layer (of the same block) via concatenation of feature maps.



The whole net looks like:

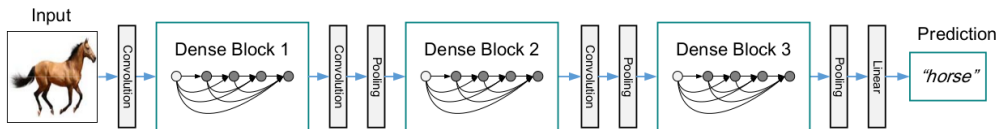


Figure 2: A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature-map sizes via convolution and pooling.

Layers	Output Size	DenseNet-121	DenseNet-169	DenseNet-201	DenseNet-264
Convolution	112×112	7×7 conv, stride 2			
Pooling	56×56	3×3 max pool, stride 2			
Dense Block (1)	56×56	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$
Transition Layer (1)	56×56	1×1 conv			
	28×28	2×2 average pool, stride 2			
Dense Block (2)	28×28	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$
Transition Layer (2)	28×28	1×1 conv			
	14×14	2×2 average pool, stride 2			
Dense Block (3)	14×14	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 24$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 64$
Transition Layer (3)	14×14	1×1 conv			
	7×7	2×2 average pool, stride 2			
Dense Block (4)	7×7	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 16$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$
Classification Layer	1×1	7×7 global average pool			
		1000D fully-connected, softmax			

Table 1: DenseNet architectures for ImageNet. The growth rate for all the networks is $k = 32$. Note that each “conv” layer shown in the table corresponds the sequence BN-ReLU-Conv.

Layers	Output Size	DenseNet-121	DenseNet-169	DenseNet-201	DenseNet-264
Convolution	112×112	7×7 conv, stride 2			
Pooling	56×56	3×3 max pool, stride 2			
Dense Block (1)	56×56	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$
Transition Layer (1)	56×56	1×1 conv			
	28×28	2×2 average pool, stride 2			
Dense Block (2)	28×28	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$
Transition Layer (2)	28×28	1×1 conv			
	14×14	2×2 average pool, stride 2			
Dense Block (3)	14×14	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 24$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 64$
Transition Layer (3)	14×14	1×1 conv			
	7×7	2×2 average pool, stride 2			
Dense Block (4)	7×7	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 16$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$
Classification Layer	1×1	7×7 global average pool			
		1000D fully-connected, softmax			

Table 1: DenseNet architectures for ImageNet. The growth rate for all the networks is $k = 32$. Note that each “conv” layer shown in the table corresponds the sequence BN-ReLU-Conv.

- Important parameter: growth rate - the number of **newly added** output channels in a convolution layer, typically small like 12,24,40
- problem: within a block that starts with k_0 channels, at depth index l one has as inputs $k_0 + (l - 1) \cdot \text{growthrate}$ many channels.

Layers	Output Size	DenseNet-121	DenseNet-169	DenseNet-201	DenseNet-264
Convolution	112×112	7×7 conv, stride 2			
Pooling	56×56	3×3 max pool, stride 2			
Dense Block (1)	56×56	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$
Transition Layer (1)	56×56	1×1 conv			
	28×28	2×2 average pool, stride 2			
Dense Block (2)	28×28	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$
Transition Layer (2)	28×28	1×1 conv			
	14×14	2×2 average pool, stride 2			
Dense Block (3)	14×14	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 24$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 64$
Transition Layer (3)	14×14	1×1 conv			
	7×7	2×2 average pool, stride 2			
Dense Block (4)	7×7	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 16$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$
Classification Layer	1×1	7×7 global average pool			
		1000D fully-connected, softmax			

Table 1: DenseNet architectures for ImageNet. The growth rate for all the networks is $k = 32$. Note that each “conv” layer shown in the table corresponds the sequence BN-ReLU-Conv.

- usually used: Densenet-BC configuration

- usually used: Densenet-BC:
 - Densenet-B: a stack of BN-ReLU-1x1 convolutions before each layer with 3x3 convolution kernels.
Reason: to reduce the # of channels and parameters in subsequent 3x3 kernels – reduction to $4 \cdot \text{growthrate}$ channels which will be the input for 3x3 convolution
 - in transition layer (see figure: where feature map size is downscaled by 1/2): 1x1 convolution reduces the number of channels to half
- low parameter count among the heavier networks, good performance

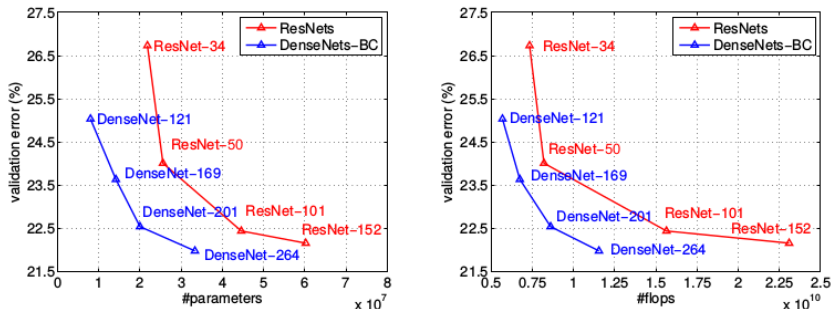


Figure 3: Comparison of the DenseNets and ResNets top-1 error rates (single-crop)

- ① Ye olde VGG
- ② Dropout
- ③ ResNets and residual connections
- ④ Batch normalization
- ⑤ DenseNets
- ⑥ Ensembles**
- ⑦ Closer to the state of the art

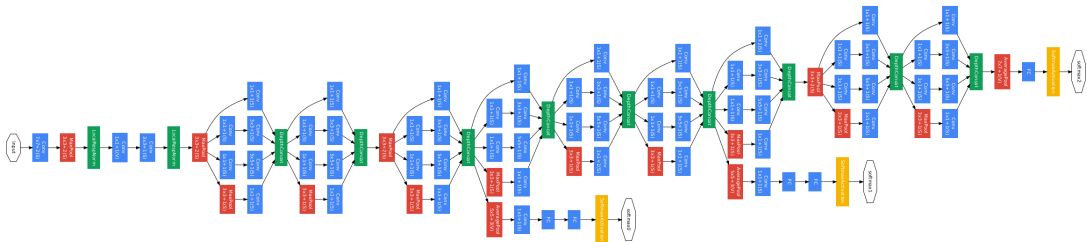


Figure 3: GoogLeNet network with all the bells and whistles

<https://arxiv.org/abs/1409.4842>

- main contribution discussed: ensembling of models
- model graphic only for eyecandy

Number of models	Number of Crops	Cost	Top-5 error	compared to base
1	1	1	10.07%	base
1	10	10	9.15%	-0.92%
1	144	144	7.89%	-2.18%
7	1	7	8.09%	-1.98%
7	10	70	7.62%	-2.45%
7	144	1008	6.67%	-3.45%

Table 3: GoogLeNet classification performance break down

<https://arxiv.org/abs/1409.4842>

- main contribution discussed: ensembling of models
- train multiple models with the same data (but different random number generator seeds)
- predict using an average taken over multiple crops and **multiple models**

$$g(x) = \frac{1}{n_m} \sum_{\text{models: } i=0}^{n_m-1} \frac{1}{n_c} \sum_{\text{crops: } k=0}^{n_c-1} f_i(\text{crop}_k(x))$$

- predict on one sample x using multiple models f_i
- average the predictions of multiple models
- use the averaged prediction

$$\widehat{E}[\{f_i, i = 0, \dots\}](x) = \frac{1}{n_m} \sum_{\text{models: } i=0}^{n_m-1} f_i(x)$$

- ① Ye olde VGG
- ② Dropout
- ③ ResNets and residual connections
- ④ Batch normalization
- ⑤ DenseNets
- ⑥ Ensembles
- ⑦ Closer to the state of the art

Basic techniques everyone should know

- Residual connections
- Batch normalization / Layer normalization
- Dropout
- using ensembles to predict

Some things which are not pretrained on even larger sets like Imagenet-22k, JFT-300M, and only use the image-net 1k:

Vision Transformers:

- (1) Swin Transformers <https://arxiv.org/abs/2103.14030> , Swin Transformers V2 <https://arxiv.org/abs/2111.09883> (relatively easy concept mod over the original ViT)
- (2) DeiT – distilled transformers <https://arxiv.org/abs/2012.12877>
- (*) not suitable for training from scratch, but good to know: the original ViT <https://arxiv.org/abs/2010.11929>
- (3) MaxVit <https://arxiv.org/abs/2204.01697v4>
- (4) DaVit <https://arxiv.org/abs/2204.03645v1>

Some things which are not pretrained on even larger sets like Imagenet-22k, JFT-300M, and only use the image-net 1k:

CNNs:

- (5) Efficientnet <https://arxiv.org/abs/1905.11946> and Efficientnet V2 <https://arxiv.org/abs/2104.00298>
- (*) notable: *normalizer-free nets* <https://arxiv.org/abs/2102.06171>
- (*) notable: *Fixing the train-test resolution discrepancy* <https://arxiv.org/abs/2003.08237v5>
- (*) notable: *Sharpness-Aware Minimization for Efficiently Improving Generalization* <https://arxiv.org/abs/2010.01412v3>
- (6) ConvNext: <https://arxiv.org/abs/2201.03545> – takes inspiration from Transformer network design